CROSS-CODE VERIFICATION

atisim against JSBSim

via the Boeing 737, at 30,000 ft and M 0.78

Two independent 6-DOF implementations are given the same coefficients at the same flight condition, and their answers are compared in four layers: the aerodynamic build-up, the trim solve, the linearisation, and a 20-second integration. A disagreement is then a defect in one of the two codes.

What this claims

atisim's aero build-up, trim solver, linearisation and integrator agree with an independent, mature engine when both are fed the same coefficients at the same state.

What this does NOT claim

That the JSBSim 737 is a correct 737. Its own file header says it was built from public data, technical reports, textbooks 'and guesses', that validation extends only to the extent that it 'seems to fly right', and that it is for 'educational and entertainment purposes only'. Nothing here supports any claim about a real 737, and none is made. What is being used is JSBSim's ENGINE, not its dataset.

Headline results

1	build-up	`CL 5.5e-09  CY 1.6e-14  Cl 1.1e-08  Cn 1.0e-08`	worst |difference|
		`CD and Cm differ; predicted to 2.9e-03`	known missing terms
2	trim	`alpha 1.981 vs 1.965 deg, thrust +1.12%`	
3	modes	`short period wn 0.04%, zeta 0.03%`	
4	trajectory	`elevator doublet 0.558 m/s over 20 s`	Earth-rotation floor 0.409

Reference: JSBSim 1.3.1 [GitHub build 1837/commit 3b25f25e49b42d0489c04ac805674fc1450ca579] May 17 2026 14:28:34

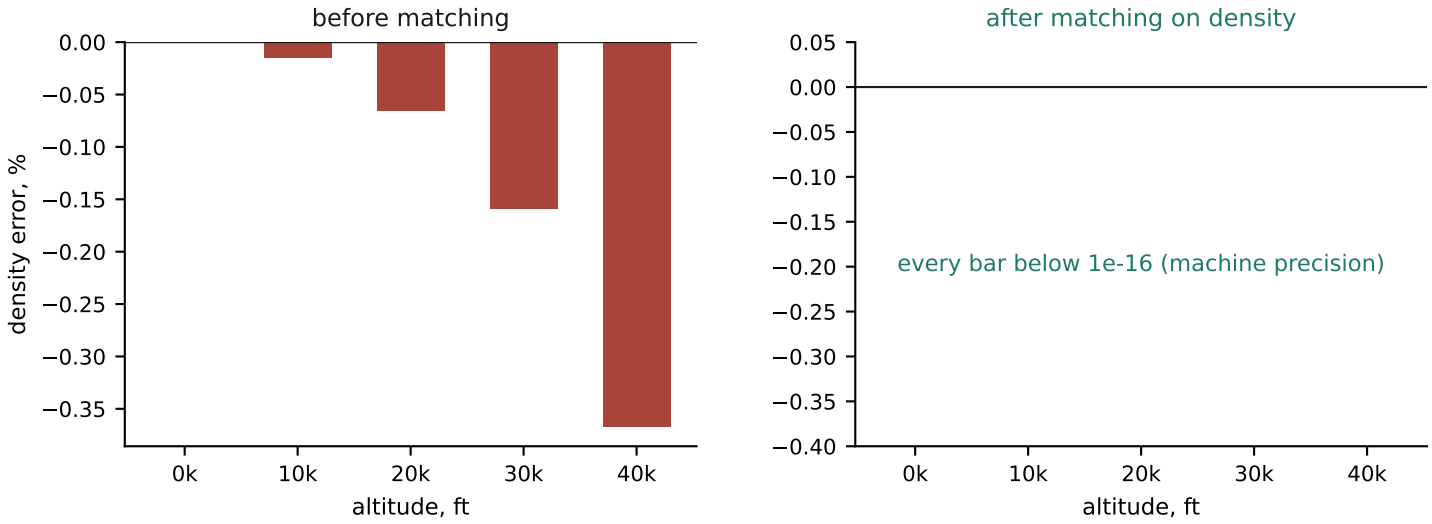
Matching the input conditions

A systematic offset in the INPUTS biases every layer in the same direction and reads as a defect in the code. Both halves were measured before any comparison was written.

Sign conventions: all sixteen agree

Measured directly against the engine rather than inferred from documentation. beta, side force, roll and yaw due to beta, aileron (mapping to the LEFT aileron position), rudder, elevator, and the wind-axis force senses all match this project's stated conventions. No sign flips are needed anywhere. JSBSim's 737 genuinely has no CYdr at all: rudder produces yaw and roll moments but zero side force, which is reproduced deliberately.

Atmosphere: one systematic error, found and neutralised



atisim's ISA uses GEOMETRIC altitude where the standard is defined on GEOPOTENTIAL altitude, so its density runs low, and the error grows with height to 0.159% at 30,000 ft and 0.367% at 40,000. Dynamic pressure is proportional to density, so that is a same-signed bias on every force in every layer -- about 22% of the layer-2 trim tolerance on its own.

Neutralised by matching density, not altitude

Altitude is not itself an input to the physics; it enters only through density and the speed of sound. The harness solves for the geometric altitude at which atisim's density equals JSBSim's -- 9130.83 m, -43.22 ft below the nominal 30,000 ft, which is exactly the geopotential correction arrived at independently. Density then agrees to 1.2e-16 relative and the speed-of-sound residual improves about ninetyfold. The underlying defect is in pre-existing code and is filed separately.

Inertia: the one convention that was assumed, and was wrong

JSBSim's ixz property is the TENSOR ELEMENT, and negating it fed the two engines different airframes: 1.6-3.2% on the lateral modes, invisible inside layer 3 old 5%. With no Cnp and no CYp, d(rdot)/dp is pure inertia coupling, so the engine settles the sign: atisim predicts +1.18079e-02, JSBSim +1.18079e-02. It was the opposite sign.

Recovering the derivatives

JSBSim applies aerodynamic forces at the AERORP (x = 625 in) but takes moments about the CG (x = 610.8 in). That 1.183 ft offset -- 0.096 of the mean chord -- is folded into every moment derivative the ENGINE exhibits, while 737.xml's <function> constants describe only the coefficient before it is applied.

	737.xml	engine	why they differ
Cma	-0.600	-0.6000	77% -- dominated by CLa x 0.096 c
Cmde	-0.849	-0.8490	CLde x 0.096 c
Clb	-0.090	-0.0900	side force 4.925 ft above the CG
Cnb	+0.260	+0.2600	side force 1.183 ft aft of the CG
CLa	+4.348	+4.3478	exact -- no offset effect on lift
Clp	-0.400	-0.4000	exact -- roll rate makes no side force
Cnr	-0.350	-0.3500	exact -- yaw rate makes no side force
Cmq	-27.0	-27.0002	= Cmq + Cmadot (-16); alphadot = q

> **Why this decided the whole design**
>
> Transcribing Cmalpha = -0.6 would have built an aeroplane with 56% of the right pitch stiffness, and the resulting mode mismatch would have been indistinguishable from a defect in dynamics.py. Differencing the RUNNING engine folds the offset in by construction, and does so in all three axes -- the roll and yaw rows above close to 0.15% and 0.17% from the same side-force-times-arm arithmetic.

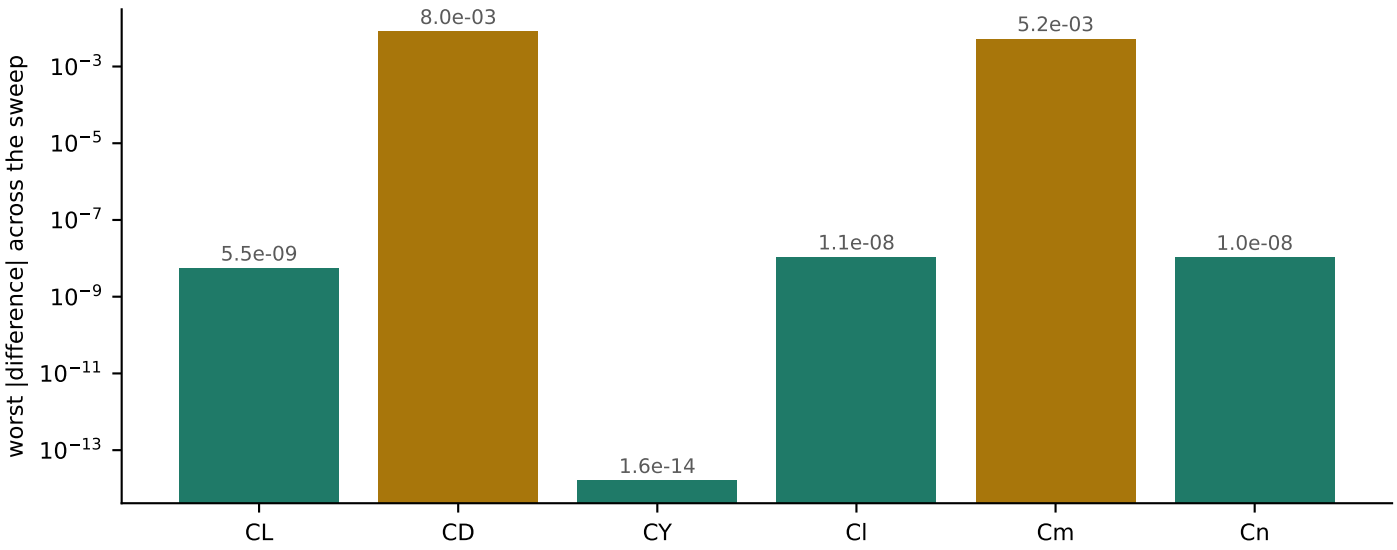
Six derivatives that are absent, and asserted to be

737.xml defines no CLq, CYp, CYr, CYdr, Cnp or Cnda. atisim carries 0.0 for each, which is agreement rather than approximation: both engines then compute the same thing. The generator MEASURES all six rather than assuming them, against a derived floor. That assertion earned its keep: at JSBSim's default 1/120 s step the settling run integrates far enough for a pitch rate to drift alpha and manufacture CLq = +4.60 out of nothing. The predicted artifact for that step size is +4.57. Settling on a 1e-6 s step instead leaves the FCS fully converged and drops the worst residual to 5.5e-04.

3 3 S W E E P P O I N T S

Layer 1 - the aerodynamic build-up

atisim's aero.coefficients against JSBSim's forces and moments at identical read-back states, sweeping alpha, beta, all three rates and all three surfaces. Driven by the recorded body-axis velocity VECTOR rather than by (V, alpha, beta), so the comparison cannot depend on how either engine defines alpha and beta.



Four of the six agree to round-off, and those are the real test of the build-up: JSBSim's CL is a table but it is linear on the segment swept here, and CY, Cl and Cn are linear in every swept variable in both engines.

CD and Cm disagree, and the disagreement is accounted for

atisim has no CD0(alpha) variation, no CDbeta and no CDde; its CD0 is those three frozen at trim. Subtracting their predicted departure from trim leaves 2.9e-03 worst case. At the sideslip points, where the raw difference is largest at 8.0e-03, the prediction accounts for it to 2.9e-10.

A prediction, not an allowance

The test asserts that the difference EQUALS the terms atisim is known to lack, computed from 737.xml's own table constants. That is a stronger statement than a tolerance: a real defect would have to disguise itself as a known missing term to survive. Cm's residual is separately shown to be quadratic about trim -- exact at the reference point to 5.9e-7 -- which is the linearisation residual from the lift vector rotating into body x at the AERORP offset, not a wrong slope.

Layer 2 - trim

JSBSim's do_simple_trim against this project's Newton solve, at the same condition and the same density.

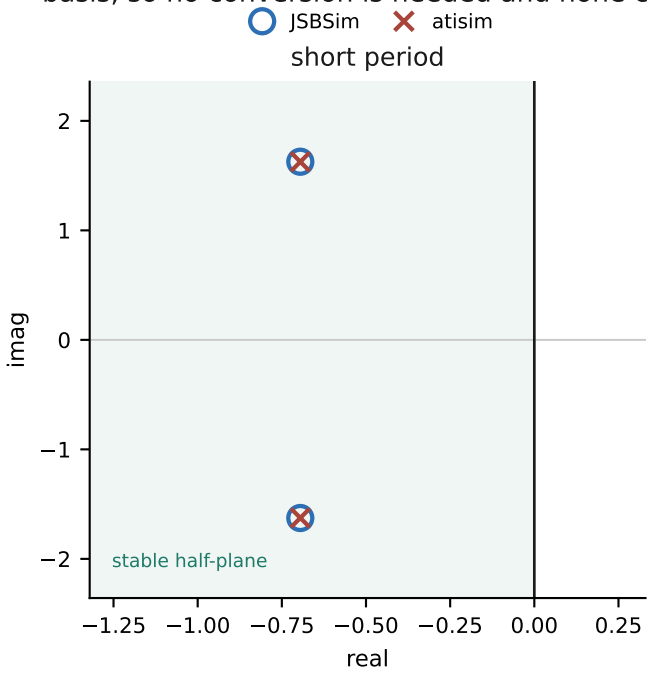
	atisim	JSBSim	difference
alpha, deg	1.98072	1.96502	+0.01571
elevator, rad	-0.053362	-0.051922	-0.001439
thrust, N	44898.4	44402.9	+1.116%
throttle	0.779502	0.771787	

The thrust difference is the linear-throttle approximation plus the ram fit residual. JSBSim blends idle and military thrust nonlinearly with throttle -- thrust per unit throttle varies 4.3x between throttle 0.2 and 1.0 -- while this model is linear in throttle, so max_thrust is fitted AT the trim throttle rather than being an engine rating.

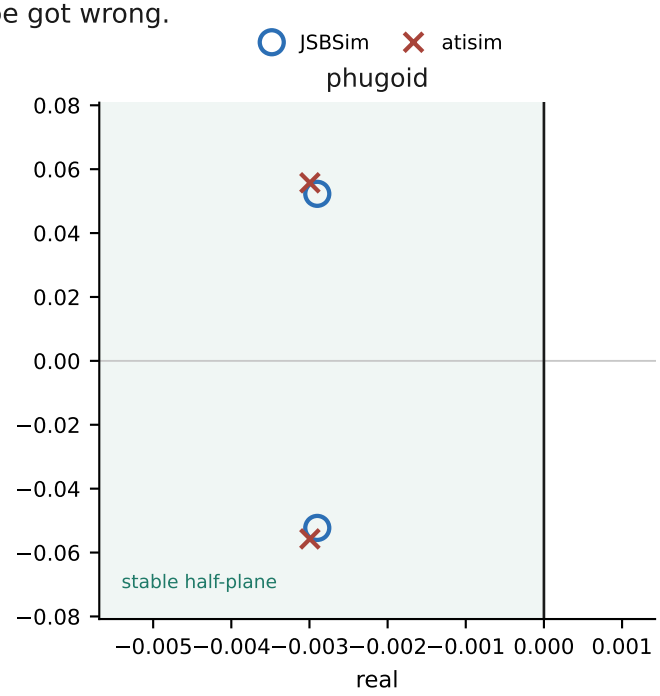
> **The turn case is recorded but not yet comparable**
>
> JSBSim also ships a steady-turn trim, and the reference holds its converged 30 deg banked solution. This project's trim.trim solves the WINGS-LEVEL problem only: its unknowns are alpha, elevator and throttle, with no bank and no aileron or rudder. The comparison is one banked-trim solver away, and the test asserts the gap rather than skipping quietly, so that adding one forces the comparison to be written.

Layer 3 - the linearisation

JSBSim exports a 12-state linear model. Its state ordering carries no labels, so the harness RE-DERIVES it from the rows of A that must be pure integrators -- theta-dot = q, psi-dot = r/cos(theta), h-dot = vt(theta - alpha) -- and fails loudly rather than silently comparing the wrong states. Modes are then compared on eigenvalues, which are invariant under the change of basis, so no conversion is needed and none can be got wrong.



real part: -0.69508 vs -0.69517



real part: -0.00299 vs -0.00290

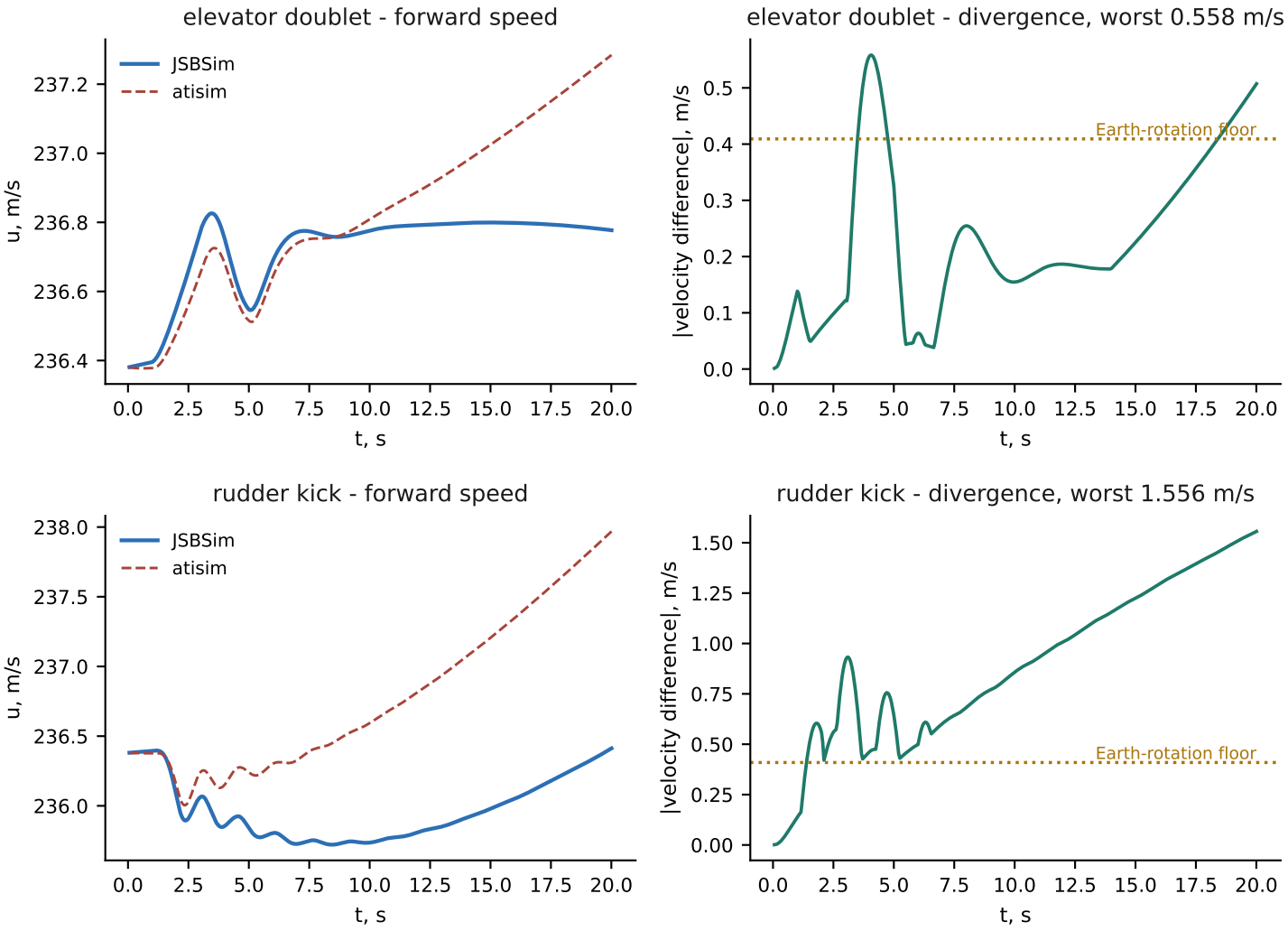
	atisim	JSBSim	difference
short period wn	1.76841	1.76918	0.044%
short period zeta	0.39305	0.39294	0.029%
phugoid wn	0.05581	0.05237	6.58%
dutch roll zeta	0.34402	0.34410	0.03%
roll TC, s	0.82847	0.82845	0.00%
spiral TC, s	16.6530	16.6911	0.23%

JSBSim's linearisation is CLOSED-LOOP

Nothing in its output says so. 737.xml's yaw damper feeds yaw rate to the rudder with unit gain above M 0.11, geared by 0.35 rad. Against the BARE airframe the lateral comparison looks catastrophic: Dutch roll zeta 0.109 against 0.344, spiral 128.4 s against 16.7 s -- a 669% disagreement that reads as broken lateral dynamics. The damper adds dCnr = Cndr x 0.35 x 2V/b = -1.147 against a bare Cnr of -0.350; folding it in closes the gap to under 2%. Pitch and roll have no such feedback, which is why the short period needs no correction at all.

Layer 4 - trajectory

Both engines integrate from the same state through the same ACHIEVED surface deflections -- not commands, so JSBSim's FCS cannot contribute to any difference.



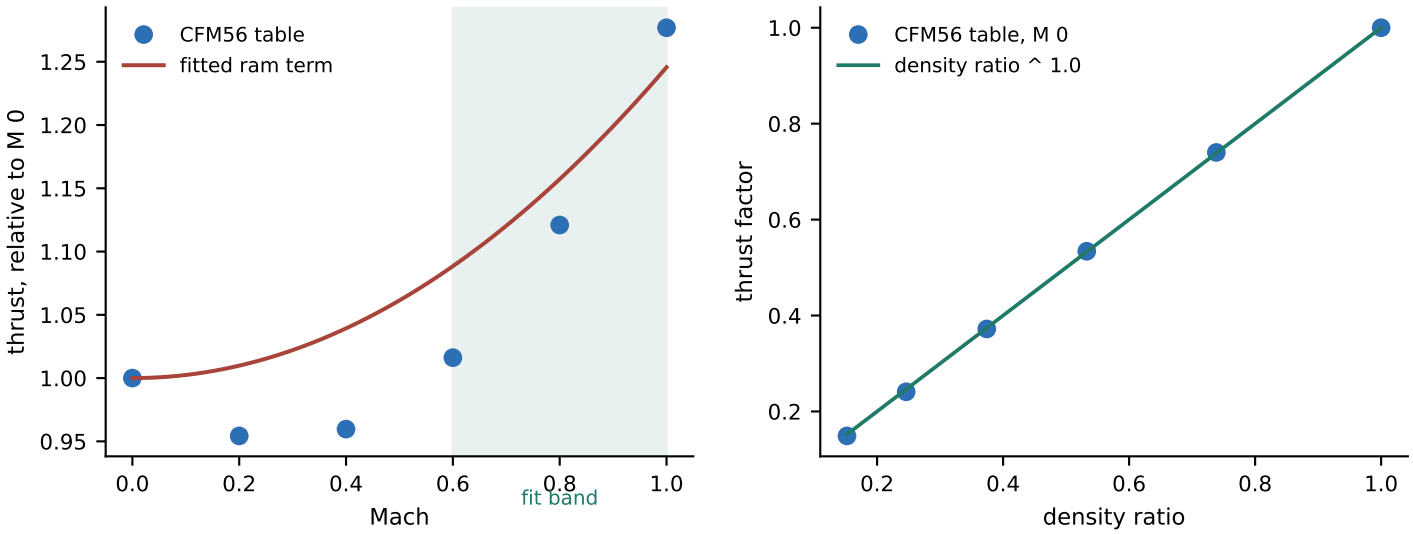
The two cases differ by 2.8x because they excite different physics, not because either is unstable. The doublet holds sideslip at 0.003 deg, so every lateral model difference is inert; the kick reaches 2.9 deg. Per component -- doublet u 0.507, v 0.012, w 0.558; kick u 1.556, v 0.933, w 0.573. Its u is secular sideslip drag, which is layer 1's missing CDbeta integrated over time.

Most of that is the replay, not the model

The replay holds each 0.05 s sample where JSBSim ran at 1/120 s. That hold is first order, so halving the rate and extrapolating removes it. What survives: doublet u 0.507 / w 0.172, kick u 1.561 / v -0.125 / w 0.530 m/s. The kick's v goes PAST zero -- no resolvable lateral residual, and reading 0.933 as Dutch-roll phase is withdrawn. The rates do the same. Left are u, secular, and w, worth 0.128 deg of equivalent alpha on the kick and 0.042 on the doublet: the same angle at both conditions, and named rather than explained.

The thrust model

atisim's thrust was throttle x maximum x density-ratio^n, with no Mach dependence at all. JSBSim's CFM56 gains 12.1% between M 0 and M 0.8 at 30,000 ft. At the M 0.78 cruise point that is an 11% thrust error, which in a trajectory comparison appears as a slow speed divergence indistinguishable from a drag defect. A Mach ram term was added.



mach_ram	0.245638	fitted over M 0.6-1.0
thrust_lapse	0.720790	fitted over 25,000-35,000 ft
max_thrust, N	101668.2	at the trim throttle, NOT a rating
fit residual, Mach	0.15%	
fit residual, altitude	1.79%	

Two things the measurement corrected

The CFM56's MilThrust table at M 0 tracks the density ratio to the power 1.0 within 0.6% -- the right panel -- so the altitude LAW was already right. But the fitted lapse at the cruise throttle is 0.72, not 1.0, because that table is full power and the idle-to-military blend at a part-throttle setting lapses differently. And max_thrust fits to about 11,700 lbf per engine against a 20,000 lbf rating, because atisim's throttle map is linear and JSBSim's is not. Neither number means what its name suggests here.

The new field defaults to a neutral value, so the 747, the 747 approach case, the Cherokee, the Cessna and the synthetic test fixture are bit-for-bit unchanged by it -- asserted in the suite rather than established by inspection.

Limitations

1. The 737 entry is valid only near cruise

The derivative set is a linearisation about 30,000 ft, M 0.78, alpha 1.97 deg. It is NOT equivalent to the project's other registry entries, which are linear derivative sets valid across the ordinary linear range. This one is a local fit to a NONLINEAR model. JSBSim's CL(alpha) peaks at 1.20 near 13 deg and falls; CL0 + CLa*alpha keeps climbing, so above about 10 deg the entry has NO stall behaviour and reports lift the source model does not have. CD0 is frozen at the trim value of a table running 0.021 to 0.042 over 0-15 deg. Flying it at 5,000 ft and 200 kt produces wrong numbers with nothing failing, warning or logging -- and it now says so: the entry carries its band in valid_mach and valid_altitude, and checks.recovery_band gates a run on it, measured in band widths outside.

2. Two of the six shipped 737 reference scripts are unused. Both are ground-roll cases and atisim has no landing-gear model, so they are unmodellable rather than merely out of scope.

3. The frozen reference can go stale silently. The suite reads the checked-in XML and never imports JSBSim, which is what makes it portable; a JSBSim release that changed a result would not be noticed until someone re-ran the generator.

4. Tolerance derivations are self-reported. Each carries its reasoning, and the CD and Cm bounds are predictions rather than allowances, but a derivation that was weak from the start would still pass its own check.

5. Defect fixing was capped to new code. The geopotential-altitude error in atmosphere.py was found here and filed separately rather than fixed, so this branch stays reviewable.

6. Agreement is not evidence about reality. Both engines can be wrong the same way, and both descend from the same textbook formulations of rigid-body flight dynamics. This catches implementation defects and says nothing about whether either models a real 737.

7. The thrust ram fit is band-limited, and wave drag is untested. The fit holds over M 0.6-1.0 and is wrong at low speed. sweep, t/c and kappa are not 737 geometry -- 737.xml gives neither sweep nor thickness -- but values placing this project's drag rise at JSBSim's tabulated onset; at M 0.78 both engines produce exactly zero wave drag, so the term is never exercised.

Rebuild: .venv/Scripts/python.exe scripts/jsbsim_report.py docs/summary/jsbsim-737-report.pdf
Regenerate the reference (needs JSBSim): python scripts/gen_jsbsim_reference.py
Every figure and number above is recomputed on each build from the frozen reference and the project's own code.